

Day 3 — Conditional Statements in Python

Detailed Seminar Notes | [if](#) • [if-else](#) • [elif](#) • [Nested if](#) • [Syntax](#) • [Rules](#) • [Examples](#) • [Viva](#)

Day 3 introduces **decision making** in Python. Until now, programs simply printed output. But real programs often need to **make decisions**: whether a student passed or failed, whether a number is even or odd, whether a person is eligible to vote, and much more. Conditional statements help us do that.

1) What are conditional statements?

Conditional statements are used when a program has to choose between different actions based on a **condition**. A condition is an expression that becomes either **True** or **False**.

For example:

- If marks are greater than or equal to 35, print **Pass**.
- If a number is divisible by 2, print **Even**.
- If age is 18 or more, print **Eligible to vote**.

2) Why do we need conditions?

- To make the program **smart** and responsive.
- To perform different actions for different situations.
- To check user input and take decisions.
- To build real-world programs like login systems, billing, grading, validation, menus, etc.

3) Types of conditional statements in Python

Statement	Use
if	Used when we want to execute a block only when a condition is True.
if-else	Used when one action should happen if condition is True and another action if it is False.
if-elif-else	Used when we have multiple conditions and multiple choices.
Nested if	Used when one if statement is written inside another if statement.

4) The if statement

The **if** statement is the simplest decision-making statement in Python. It runs a block of code **only if the condition is True**.

Syntax:

```
if condition:
    statement(s)
```

Important points:

- The condition is written after **if**.
- A **colon :** is compulsory after the condition.
- The statements inside the if block must be **indented**.

- If the condition is False, the if block is skipped.

Example 1: Check positive number

```
num = 10
if num > 0:
    print("Positive number")
```

Since **10 > 0** is True, the output is **Positive number**.

Example 2: Check if marks are pass

```
marks = 75
if marks >= 35:
    print("Pass")
```

5) Flow of an if statement

Step	What happens
1	Python checks the condition.
2	If condition is True, the indented block runs.
3	If condition is False, the block is skipped and program continues.

6) The if-else statement

The **if-else** statement is used when the program must choose between **two alternatives**.

Syntax:

```
if condition:
    statement(s)    # runs if condition is True
else:
    statement(s)    # runs if condition is False
```

Example 1: Even or odd

```
num = 7
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Since **7 % 2 == 0** is False, the output is **Odd**.

Example 2: Pass or Fail

```
marks = 28
if marks >= 35:
    print("Pass")
else:
    print("Fail")
```

7) Difference between if and if-else

if	if-else
Runs code only when condition is True.	Chooses between two blocks.
If condition is False, nothing happens in that block.	If condition is False, else block runs.
Useful when action is needed only for True case.	Useful when both True and False cases need actions.

8) The if-elif-else statement

When we have **more than two choices**, we use **if-elif-else**. The word **elif** means **else if**.

Syntax:

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
elif condition3:
    statement(s)
else:
    statement(s)
```

Python checks the conditions from top to bottom. As soon as one condition becomes True, that block executes and the remaining conditions are skipped.

Example 1: Grade calculator

```
marks = 82

if marks >= 90:
    print("Grade A+")
elif marks >= 75:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
elif marks >= 35:
    print("Grade C")
else:
    print("Fail")
```

Here, marks = 82, so **marks >= 75** is True and the output becomes **Grade A**.

Example 2: Find largest of 3 numbers

```
a = 10
b = 25
c = 18

if a > b and a > c:
    print("a is largest")
elif b > a and b > c:
    print("b is largest")
else:
    print("c is largest")
```

9) Nested if statement

A **nested if** means writing one **if** statement inside another **if** statement.

Syntax:

```
if condition1:
    statement(s)
    if condition2:
        statement(s)
```

Example: Eligible for scholarship

```
marks = 88
income = 20000

if marks >= 80:
```

```
print("Academic condition passed")
if income < 30000:
    print("Eligible for scholarship")
```

The second if is checked only when the first if condition is True.

10) Conditions and comparison operators

Conditions are usually created using **comparison operators**.

Operator	Meaning	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	marks >= 35
<=	Less than or equal to	age <= 18

11) Logical operators in conditions

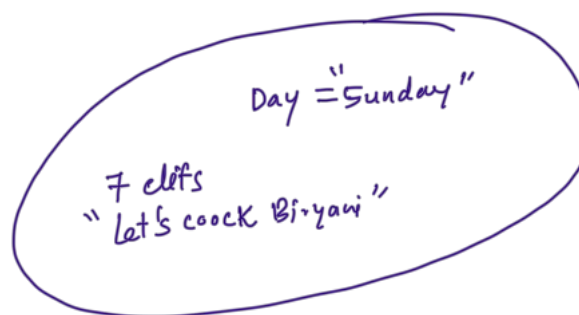
Sometimes one condition is not enough. We may need to combine conditions using logical operators.

Operator	Meaning	Example
and	True only if both conditions are True	age >= 18 and citizen == True
or	True if at least one condition is True	marks >= 35 or sports_quota == True
not	Reverses the result	not(a > b)

Example using logical operators

```
age = 20
has_id = True

if age >= 18 and has_id == True:
    print("Allowed to enter")
else:
    print("Not allowed")
```



12) Important rules for conditional statements

- Always write a **colon :** after **if**, **elif**, and **else**.
- Indent the statements inside the block.
- Use **==** for comparison, not **=**.
- Conditions must be meaningful and should return **True** or **False**.
- In **if-elif-else**, Python checks from top to bottom and stops at the first True condition.
- Use nested if only when it improves logic clarity.

13) Common beginner mistakes

- Using **=** instead of **==** inside conditions.
- Forgetting the colon after if or else.
- Wrong indentation of the block.
- Writing else with a condition. Else never takes a condition.
- Placing broader conditions before narrower ones in if-elif-else, which may skip correct grading logic.

14) Dry run example — even or odd

```
num = 8
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Step	Value / Check	Result
1	num = 8	Stored
2	8 % 2 == 0	True
3	if block runs	Prints "Even"
4	else block skipped	-

15) Important beginner programs

Program 1: Check positive, negative, or zero

```
num = int(input("Enter a number: "))

if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")
```

Program 2: Check voting eligibility

```
age = int(input("Enter age: "))

if age >= 18:
    print("Eligible to vote")
else:
```

```
print("Not eligible to vote")
```

Program 3: Check largest of 2 numbers

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

if a > b:
    print("First number is larger")
else:
    print("Second number is larger")
```

Program 4: Grade using marks

```
marks = int(input("Enter marks: "))

if marks >= 90:
    print("A+")
elif marks >= 75:
    print("A")
elif marks >= 60:
    print("B")
elif marks >= 35:
    print("C")
else:
    print("Fail")
```

16) Real-life uses of conditions

- Login validation — if password is correct, allow access.
- ATM system — if balance is enough, allow withdrawal.
- College portal — if attendance is above limit, allow exam hall ticket.
- E-commerce — if total purchase is above a value, apply discount.
- Games — if player health becomes zero, show Game Over.

17) How to explain Day 3 in a seminar

- Start with the meaning of decision making in programming.
- Explain what a condition is and that it returns True or False.
- Teach **if** first with a simple example.
- Then explain **if-else** using even/odd or pass/fail.
- After that explain **if-elif-else** with grading logic.
- Finally explain nested if and logical operators.
- End with common mistakes and one or two practice programs.

18) Viva / seminar questions

- What is a conditional statement?
- What is the use of if statement?
- What is the difference between if and if-else?
- Why do we use if-elif-else?

- What is nested if?
- What is the difference between = and == ?
- Can else have a condition?
- What are logical operators in Python?
- How does Python decide which elif block to execute?
- Why is indentation important in conditions?

19) Short seminar summary

Conditional statements are used to make **decisions** in Python. The main forms are **if**, **if-else**, **if-elif-else**, and **nested if**. Conditions are built using comparison and logical operators. These statements help programs choose the correct action based on user input or data values.

20) Quick revision box

Topic	Key point
if	Runs block only when condition is True
if-else	Chooses between two blocks
if-elif-else	Used for multiple conditions / multiple choices
Nested if	if statement inside another if
Comparison operators	==, !=, >, <, >=, <=
Logical operators	and, or, not